Проект по ИИ-мышлению: ассистент сервисной службы

Выполнил: Юманов Михаил

7 ноября 2026 г.

Репозиторий: `https://github.com/MikhailIum/llm-project`

1 Задача

Диспетчеру сервисной службы нужно по модели оборудования и коду ошибки быстро определить неисправный узел, установленную деталь, её актуальную замену, поставщика, а также наличие и цену на складе. Необходимые сведения раскиданы по разным документам компании: руководствам, бюллетеням и отчётам, а текущие остатки компонентов и цены доступны через запрос к базе ERP. Диспетчер тратит на каждую итерацию около 12 минут. В связи с этим предлагается разработать ассистента, который будет отвечать на запросы диспетчера на основе документов и обращения к системе ERP.

Изначально в системе реализован плоский RAG. Рассмотрим запрос: «Ошибка E-17 на AR-210. Какую деталь заказать, есть ли она на складе в Казани и сколько она стоит?». Плоский RAG нашёл упоминание ошибки и узла, но не назвал код детали, остаток или цену. В выдачу попали текстовые фрагменты, но поиск не собрал цепочку сведений из руководства и бюллетеня. Поэтому предложено реализовать Graph RAG, который позволяет пройти цепочку `E-17` → `пусковое реле` → `P-1042` → `P-1042B`, а в источниках указан бюллетень. Однако он не имеет доступа к ERPи не может сообщить остаток и цену. Поэтому предложено реализовать агента, который пройдет цепочку `E-17` → `пусковое реле` → `P-1042` → `P-1042B`, вызвав локальный поиск, проверит остаток и цену через запрос к ERP и ответит: 3 штуки в Казани, 2630 рублей.

2 Решение

2.1 Индексация

Корпус документов разбивается на чанки. Для чанков вызывается извлечение сущностей и направленных отношений по заданной схеме, коды нормализуются, названия приводятся к словарному виду. При сборке индекса результаты объединяются в ориентированный граф, в котором узлы содержат тип и имя сущности, а также источники, в которых она упоминается, а ребра – тип отношения между двумя сущностями, вес по числу чанков, где это отношение описано.

Для глобального поиска граф разбивается на сообщества алгоритмом Лувена. Модель генерирует краткие сводки сообществ, а числовые характеристики предварительно рассчитываются кодом.

2.2 Два режима поиска и агент

- **Локальный.** Алгоритм находит в графе опорные коды и названия. Для нескольких опорных сущностей выбираются пути между ними не длиннее трёх ребер без учёта направления, для одной – окрестность до двух связей. Далее добавляются цепочки `REPLACED_BY`, чтобы прийти к итоговой детали. Эти данные вместе с вопросом передаются модели.
- **Глобальный.** Сводки сообществ обрабатываются в два этапа. Сначала модель возвращает частичные ответы для каждого сообщества с оценкой релевантности, затем ответы с положительной оценкой сводятся в итог (парадигма map-reduce).
- **Агент.** Агент использует поиск по базе знаний и пять инструментов, включая проверку остатка, цены и создание заказа. Код проверяет параметры ERP, подтверждённость сведений о детали и модели, допустимость склада, явное намерение заказать и наличие количества в вопросе. Заказ при обычном режиме подготавливается к подтверждению, а не создаётся автоматически.

Примеры принятых решений: увеличение окна генерации при извлечении, чтобы помещались связи из больших чанков, и добавление в промпт примеров с разными типами отношений из документов вне тестовой выборки.

3 Результаты

Модель: `qwen3:4b-instruct-2507-q4_K_M`

Режим	Acc	Local	Multi	Global	ERP	Refusal	Source Recall	Fabricated
Плоский RAG	50.0%	100%	57.1%	0%	0%	100%	85.6%	0%
Graph local	62.5%	100%	57.1%	75%	0%	100%	88.1%	0%
Graph global	25.0%	60%	0%	25%	0%	66.7%	50.0%	0%
Agent	58.3%	100%	57.1%	0%	60%	66.7%	88.1%	0%

Режим	Вызовов / вопрос	Токенов / вопрос	Tools rejections
Плоский RAG	1.000	1 358	0%
Graph local	0.833	967	0%
Graph global	8.458	3 927	0%
Агент	4.542	3 877	3.6%

Извлечение: Precision сущностей – 1.000, Recall – 0.982. Precision связей – 1.000, Recall – 0.932. Проверки отбросили 27 связей при полной индексации корпуса из 80 документов и 9 – на тестовых документах. Полная индексация в локальном режиме потребовала 100 вызовов модели и 167 523 токена (121 597 входных и 45 926 выходных). Построены граф из 118 узлов и 261 ребра и 8 сообществ.

4 Где граф помогает и где уступает

Помогает. На вопрос «Ошибка E-17 на AR-210. Какую деталь заказать, есть ли она на складе в Казани и сколько она стоит?» плоский RAG нашёл упоминание ошибки и пускового реле, но не назвал деталь, остаток или цену. Graph local прошёл по связям от ошибки через узел и замену и назвал P-1042B, остаток и цену он получить не мог, поскольку в этом режиме нет доступа к ERP. Агент по той же цепочке вызвал `check_stock` и `get_price` и вернул «3 штуки» и «2630 рублей».

Уступает. На вопрос «Какая деталь сейчас ставится в BZ-110 как узел “нагреватель оттайки”?» плоский RAG ответил P-2054C, связав два бюллетеня о последовательных заменах. Graph local неверно принял связь `BZ-110 -FAILED-> нагреватель оттайки` за ответ о детали, Graph global отказал. Агент тоже повторил ошибку local и назвал «нагреватель оттайки». С глобальным поиском решение справилось посредственно – маленькая модель плохо обобщает кластеры.

5 Ошибки

Диагностические запросы повторно выполнены после пересборки индекса текущим кодом. В таблице разделены ошибки разных этапов и последствия пропуска данных на предыдущем этапе.

Этап	**Пример и наблюдение**	**Причина**
Извлечение	В руководстве BZ-110 перечислена P-2010 как деталь компрессора. На вопрос «Какая деталь установлена в BZ-110?» режим local отвечает отказом. При извлечении из этого фрагмента модель выдала связь с обратным направлением `Part -HAS_PART-> Model`.	Схема допускает только `Model -HAS_PART-> Part`; проверка отбрасывает связь с неверным направлением, поэтому факт не попадает в граф и поиск отказывает. Это одна из причин, по которым при полной индексации проверки извлечения отбросили 27 связей.
Local retrieval	На вопрос «Кто поставляет актуальную деталь для ошибки E-14 на AR-320?» local ответил, что поставщик неизвестен. В графе есть `P-1118B -SUPPLIED_BY-> ТермоПрибор`; связь подтверждена бюллетенем SB-2025-02. Если указать P-1118B прямо в вопросе, local называет ТермоПрибор.	При опорных сущностях E-14 и AR-320 поиск выбирает пути между ними и цепочки замен, но не включает нужное ребро поставщика от последней детали цепочки.
Сообщества и сводки	В пересобранном графе 81 из 261 ребра соединяет разные сообщества, в том числе 60 `HAS_PART`, 11 `FAILED` и 2 `REPLACED_BY`. Например, цепочка P-2054 → P-2054B разделена между сообществами.	При составлении сводки учитываются только рёбра, у которых оба конца принадлежат тому же сообществу. Межсообщественные связи и части цепочек замен поэтому не описываются.
Global map/reduce	На вопрос «Какой узел чаще всего отказывает в линейке “Арктика”?» global сообщил, что частоты неизвестны, хотя сумма весов `FAILED` в графе равна 9 для пускового реле, 4 для датчика температуры и 1 для контроллера.	При map/reduce теряются межсообщественные связи и детали цепочек замен; сводки также не сохраняют достаточно информации о весах `FAILED`.

6 Границы системы

Сведения об остатке и цене берутся из ERP, а не выводятся из документов. Создание заказа требует явной просьбы пользователя, подтверждённых кодов детали и модели, допустимого склада и количества, указанного пользователем. При стандартном `confirm=False` функция агента не вызывает инструмент создания заказа: она возвращает `needs_confirmation=True` и `pending_action`. Только повторный запуск с явным `confirm=True` разрешает создание. Цикл агента ограничен шестью вызовами инструментов по умолчанию, нечитаемые аргументы, неизвестные инструменты, неподтверждённые номера деталей и неуказанное количество отклоняются или запрашиваются у пользователя. Повторный вызов с теми же аргументами использует сохранённый результат. Проверки кодов и аргументов выполняются до вызова ERP, при её недоступности запрос повторяется до двух раз. Ошибка ERP и отсутствие записи отличаются от успешного ответа с нулевым остатком.

7 Следующий шаг

Можно было бы попробовать добавить сводку/статистику для связей между сообществами. Так как много полезной информации может теряться, таких связей много, как показали эксперименты.

8 Вклад

Я выполнял проект один. Использовал агента, о котором в следующем пункте.

9 Использование ИИ-инструментов

Использовался агент Copilot. Просил его написать код каждого отдельного модуля и использовал его как основу. Проверял каждую строку и изменял то, что я сам написал бы иначе. В итоге почти весь код переписан. Использовал нейронку для генерации хороших промптов. Просил найти узкие места в коде на основе логов и дополнительных тестов, подготовить «рыбу» отчёта.

Вердикт самого агента: Автор проверял требования и принимал решения о том, какие изменения включать. ИИ-инструменты применялись как вспомогательные: для сверки реализации с требованиями, диагностики ошибок, выполнения отдельных согласованных правок и подготовки/запуска регрессионных тестов.